

Информация

Докладчик

.....: {column align=center}

::: {column width="70%"}

- Эспиноса Василита Кристина Микаела
- студентка
- Российский университет дружбы народов
- 1032224624@pfur.ru
- <https://github.com/crisespinosal/>

:::

::: {column width="30%"}

:::

.....:

Цель работы

Реализовать модель $M|M|1$ в CPN tools

Задание

- Реализовать в CPN Tools модель системы массового обслуживания $M|M|1$.
- Настроить мониторинг параметров моделируемой системы и нарисовать графики очереди.

Выполнение лабораторной работы

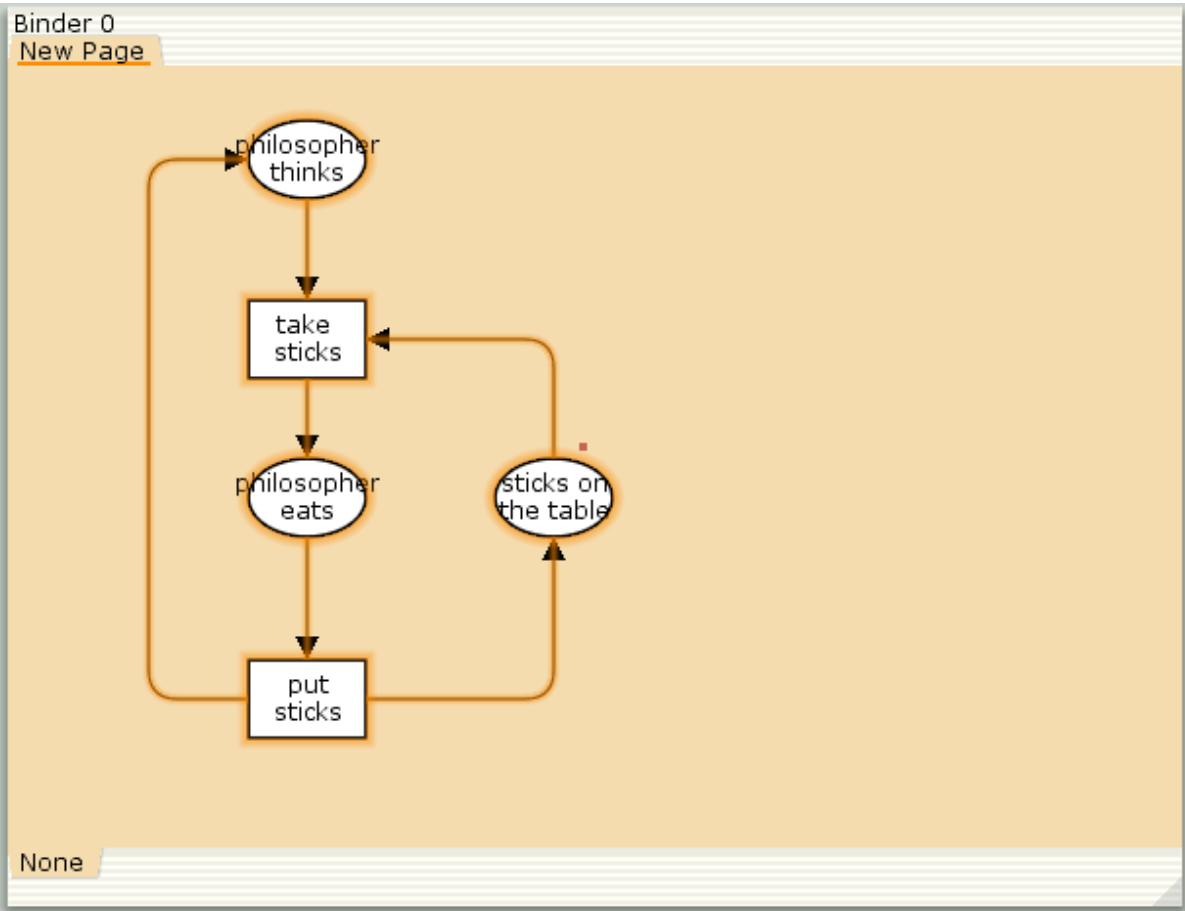
В систему поступает поток заявок двух типов, распределённый по пуассоновскому закону. Заявки поступают в очередь сервера на обработку. Дисциплина очереди - FIFO. Если сервер находится в режиме ожидания (нет заявок на сервере), то заявка поступает на обработку сервером.

Будем использовать три отдельных листа: на первом листе опишем граф системы (рис. [-@fig:001]), на втором — генератор заявок (рис. [-@fig:002]), на третьем — сервер обработки заявок (рис. [-@fig:003]).

Сеть имеет 2 позиции (очередь — Queue, обслуженные заявки — Complited) и два перехода (генерировать заявку — Arrivals, передать заявку на обработку серверу — Server). Переходы имеют сложную иерархическую структуру, задаваемую на отдельных листах модели (с помощью соответствующего инструмента меню — Hierarchy).

Выполнение лабораторной работы

Между переходом Arrivals и позицией Queue, а также между позицией Queue и переходом Server установлена дуплексная связь. Между переходом Server и позицией Complited — односторонняя связь.

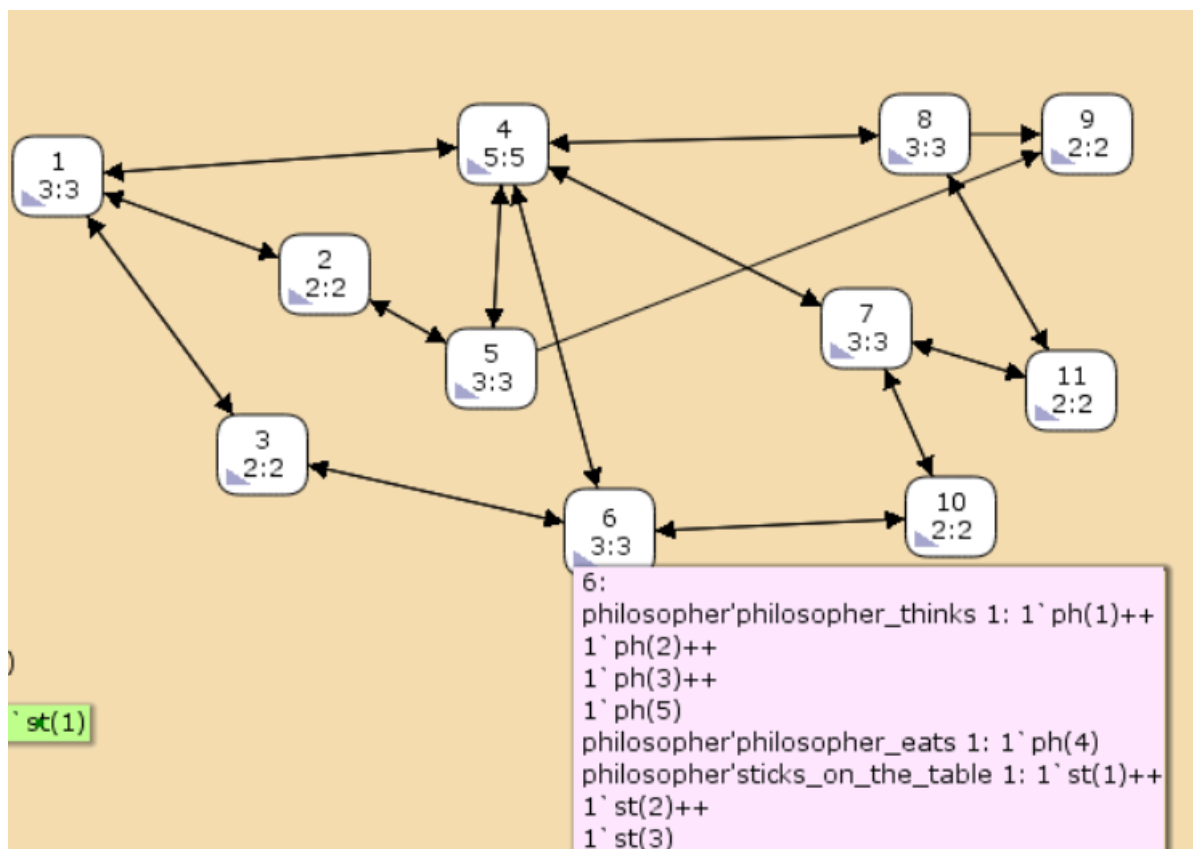


Выполнение лабораторной работы

Граф генератора заявок имеет 3 позиции (текущая заявка — Init, следующая заявка — Next, очередь — Queue из листа System) и 2 перехода (Init — определяет распределение поступления заявок по экспоненциальному закону с интенсивностью 100 заявок в единицу времени, Arrive — определяет поступление заявок в очередь).



Граф процесса обработки заявок на сервере имеет 4 позиции (Busy — сервер занят, Idle — сервер в режиме ожидания, Queue и Complited из листа System) и 2 перехода (Start — начать обработку заявки, Stop — закончить обработку заявки).



Выполнение лабораторной работы

Зададим декларации системы (рис. [-@fig:004]).

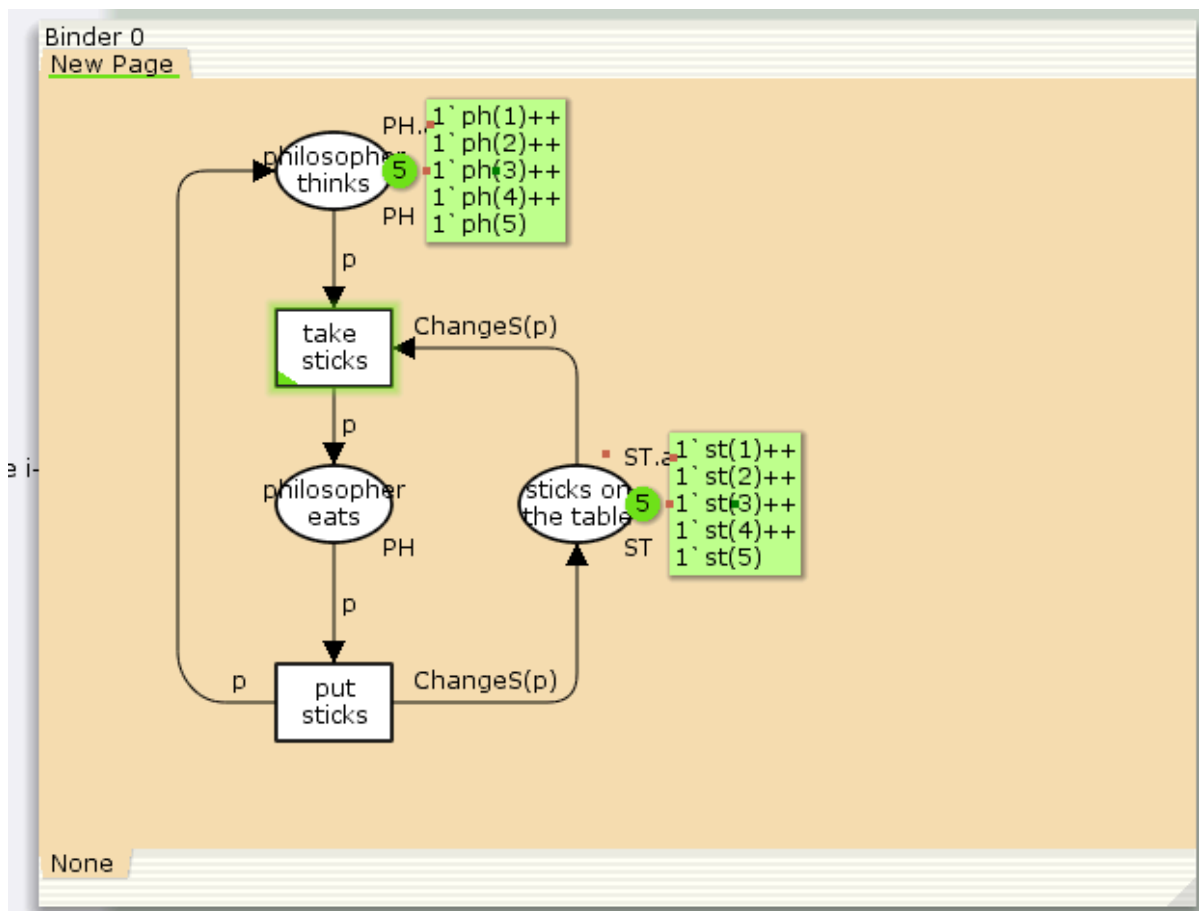
Определим множества цветов системы (colorset):

```
history
▼ Declarations
  ▼ Standard declarations
    ▶ colset UNIT
    ▶ colset INT
    ▶ colset BOOL
    ▶ colset STRING
  ▼ val n = 5;
  ▼ colset PH = index ph with 1..n;
  ▼ colset ST = index st with 1..n;
  ▼ var p:PH;
  ▼ fun ChangeS (ph(i)) =
    1`st(i)++ 1`st(if i = n then 1 else i+1)
  ▶ Monitors
  New Page
```

Выполнение лабораторной работы

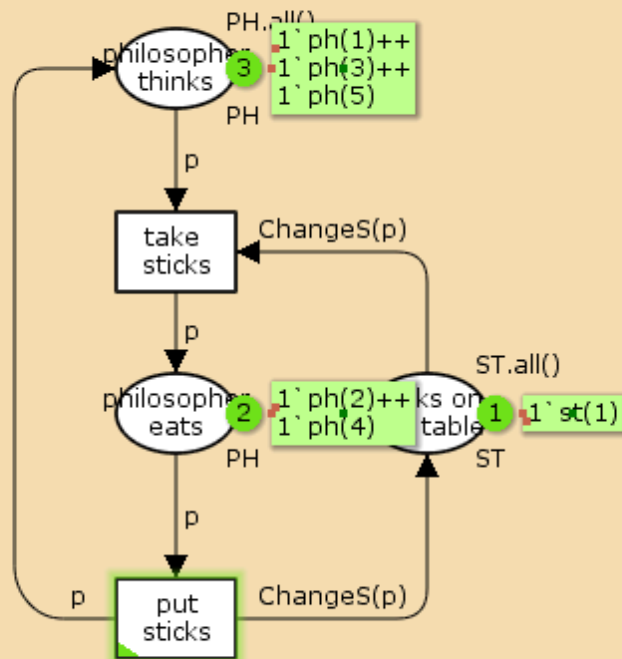
Зададим параметры модели на графах сети.

На листе System (рис. [-@fig:005]):



Выполнение лабораторной работы

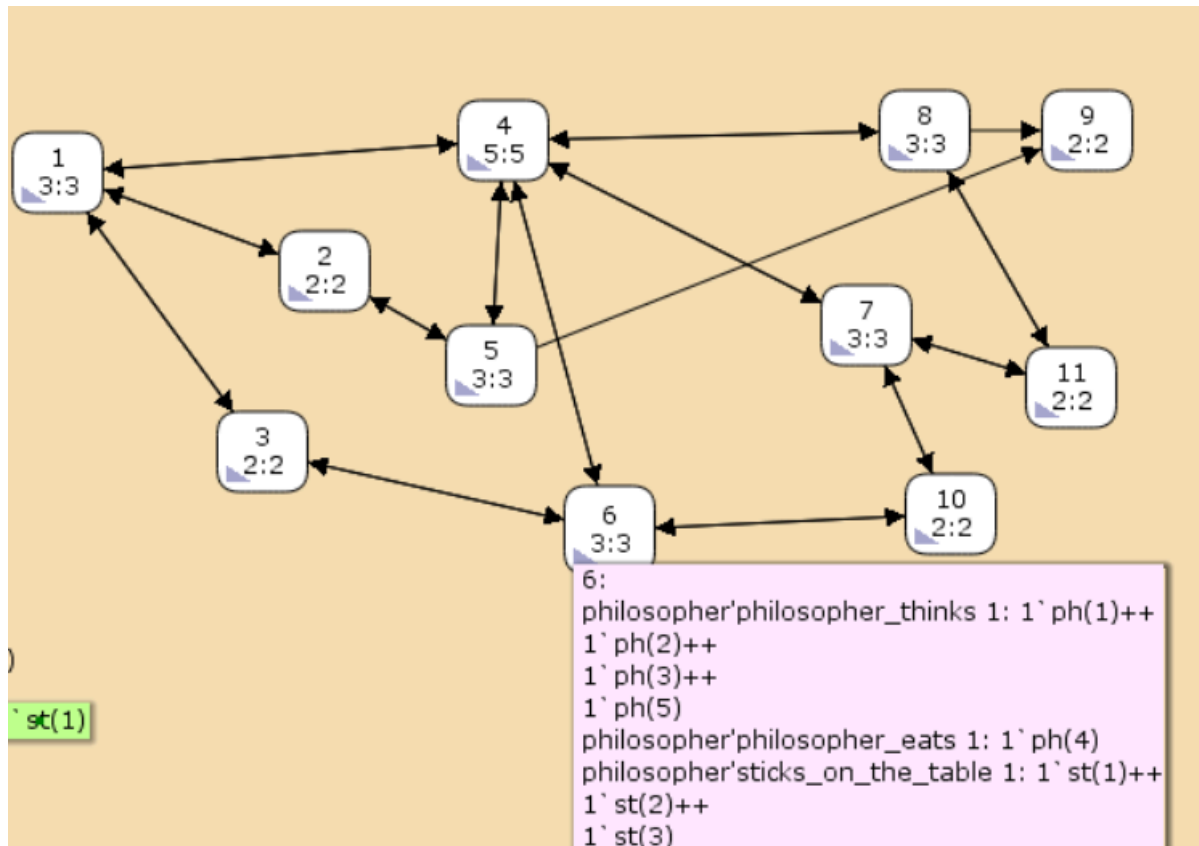
На листе Arrivals (рис. [-@fig:006]):



None

Выполнение лабораторной работы

На листе Server (рис. [-@fig:007]):



Мониторинг параметров моделируемой системы

Потребуется палитра Monitoring. Выбираем Break Point (точка останова) и устанавливаем её на переход Start. После этого в разделе меню Monitor появится новый подраздел, который назовём Ostanovka. В этом подразделе необходимо внести изменения в функцию Predicate, которая будет выполняться при запуске монитора. Зададим число шагов, через которое будем останавливать мониторинг. Для этого true заменим на Queue_Delay.count()=200.

```
Binder 0
Arrivals System Server fun pred <Ostanovka>
fun pred (bindelem) =
let
  fun predBindElem (Server'Start (1,
                                {job,jobs,proctime})) = Queue_Delay.count()=200
  | predBindElem _ = false
in
  predBindElem bindelem
end
```

Мониторинг параметров моделируемой системы

Необходимо определить конструкцию Queue_Delay.count(). С помощью палитры Monitoring выбираем Data Call и устанавливаем на переходе Start. Появившийся в меню монитор называем Queue Delay (без подчеркивания). Функция Observer выполняется тогда, когда функция предикатора выдаёт значение true. По умолчанию функция выдаёт 0 или унарный минус (~1), подчёркивание обозначает произвольный аргумент. Изменим её так, чтобы получить значение задержки в очереди. Для этого необходимо из текущего времени intTime() вычесть временную метку AT, означающую приход заявки в очередь.

```
Binder 0
Arrivals System Server fun obs <Queue Delay>
fun obs (bindelem) =
let
  fun obsBindElem (Server'Start (1, {job,jobs,proctime})) =
(intTime() - (#AT job))|
  | obsBindElem _ = ~1
in
  obsBindElem bindelem
end
```

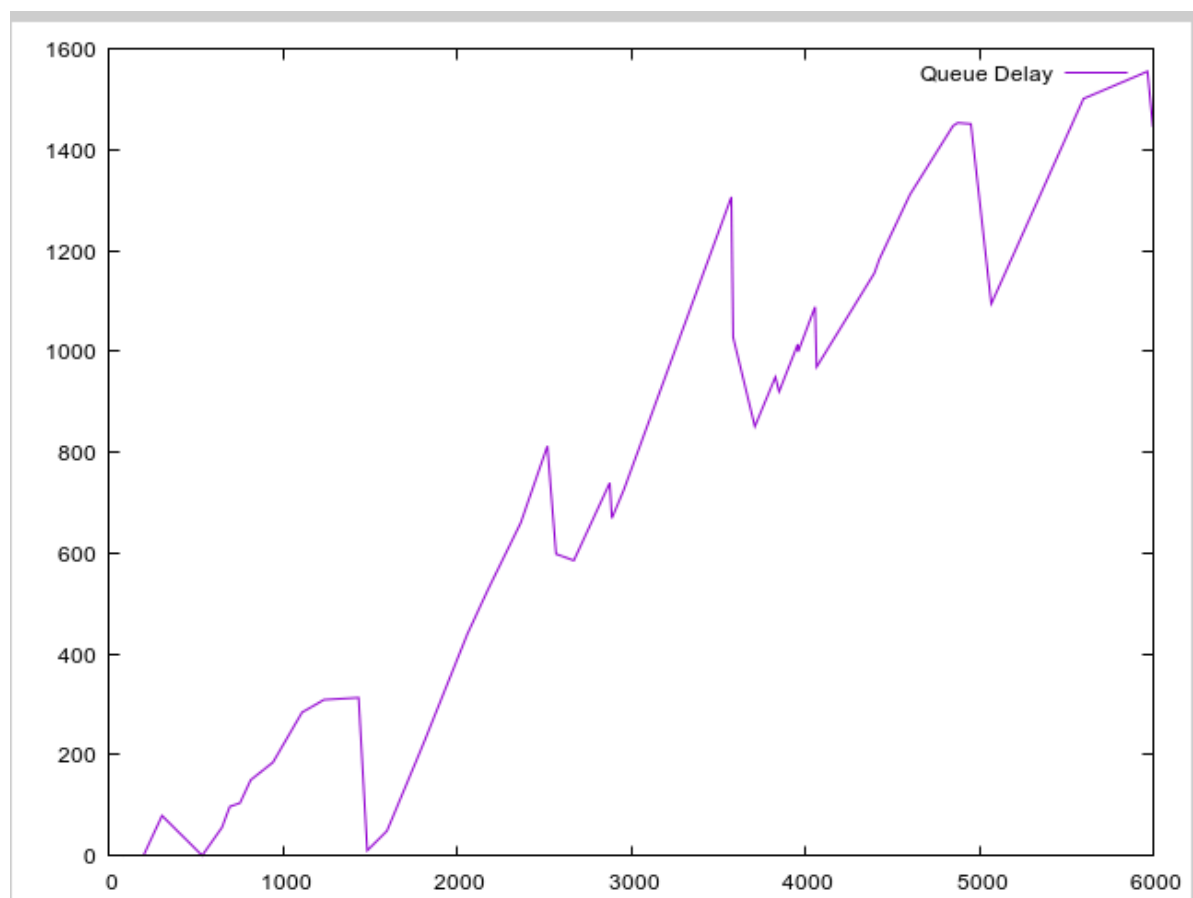
Мониторинг параметров моделируемой системы

После запуска программы на выполнение в каталоге с кодом программы появится файл Queue_Delay.log (рис. [-@fig:010]), содержащий в первой колонке — значение задержки очереди, во второй — счётчик, в третьей — шаг, в четвёртой — время.

Файл	Правка	Поиск	Вид	Документ	Спр
#data counter step time					
0	1	3	496		
0	2	6	507		
0	3	9	523		
97	4	22	624		
134	5	27	662		
266	6	45	808		
308	7	50	850		
356	8	57	913		
347	9	60	915		
341	10	63	920		
501	11	78	1085		
664	12	97	1256		

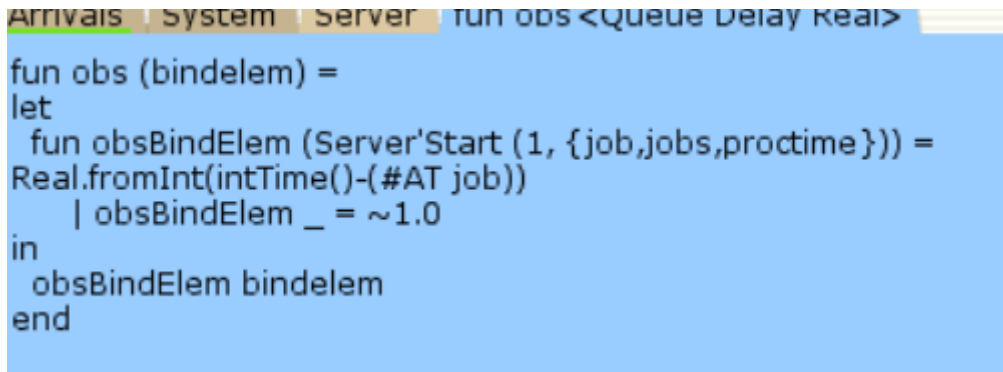
Мониторинг параметров моделируемой системы

С помощью gnuplot можно построить график значений задержки в очереди (рис. [-@fig:011]), выбрав по оси x время, а по оси y — значения задержки:



Мониторинг параметров моделируемой системы

Посчитаем задержку в действительных значениях. С помощью палитры Monitoring выбираем Data Call и устанавливаем на переходе Start. Появившийся в меню монитор называем Queue Delay Real. Функцию Observer изменим следующим образом(рис. [-@fig:012]):



```
fun obs (bindelem) =  
let  
  fun obsBindElem (Server'Start (1, {job,jobs,proctime})) =  
    Real.fromInt(intTime()-(#AT job))  
    | obsBindElem _ = ~1.0  
in  
  obsBindElem bindelem  
end
```

Мониторинг параметров моделируемой системы

По сравнению с предыдущим описанием функции добавлено преобразование значения функции из целого в действительное, при этом obsBindElem _ принимает значение ~1.0. После запуска программы на выполнение в каталоге с кодом программы появится файл Queue_Delay_Real.log с содержимым, аналогичным содержимому файла Queue_Delay.log, но значения задержки имеют действительный тип (рис. [-@fig:013]):

Файл	Правка	Поиск	Вид	Документ	Справка
#data	counter	step	time		
0.000000	1	3	69		
60.000000	2	6	197		
135.000000	3	9	373		
75.000000	4	12	499		
0.000000	5	15	777		
12.000000	6	18	844		
0.000000	7	21	957		
175.000000	8	27	1167		
397.000000	9	30	1419		
459.000000	10	34	1537		
647.000000	11	39	1726		
536.000000	12	41	1879		
471.000000	13	43	1929		
690.000000	14	47	2201		
626.000000	15	49	2219		
626.000000	16	51	2231		
633.000000	17	54	2276		
358.000000	18	56	2289		
260.000000	19	58	2304		
650.000000	20	70	2924		
586.000000	21	72	2934		
709.000000	22	74	3059		
821.000000	23	76	3177		
714.000000	24	78	3198		

Мониторинг параметров моделируемой системы

Посчитаем, сколько раз задержка превысила заданное значение. С помощью палитры Monitoring выбираем Data Call и устанавливаем на переходе Start. Монитор называем Long Delay Time. Функцию Observer изменим следующим образом (рис. [-@fig:014]):

```

Binder 0
Arrivals System Server fun obs <Long Delay Time>
fun obs (bindelem) =
if IntInf.toInt(Queue_Delay.last()) >= (!longdelaytime)
then 1
else 0

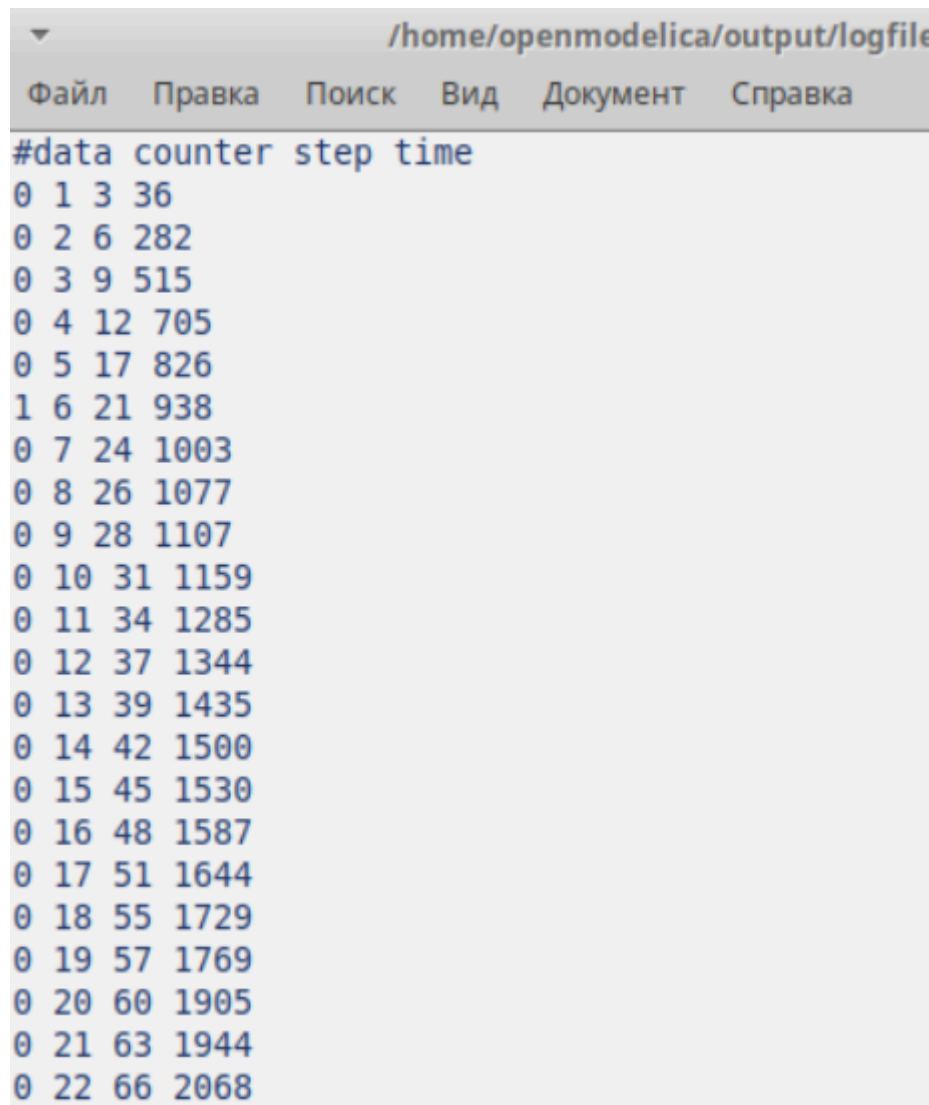
```

Мониторинг параметров моделируемой системы

При этом необходимо в декларациях задать глобальную переменную (в форме ссылки на число 200): longdelaytime

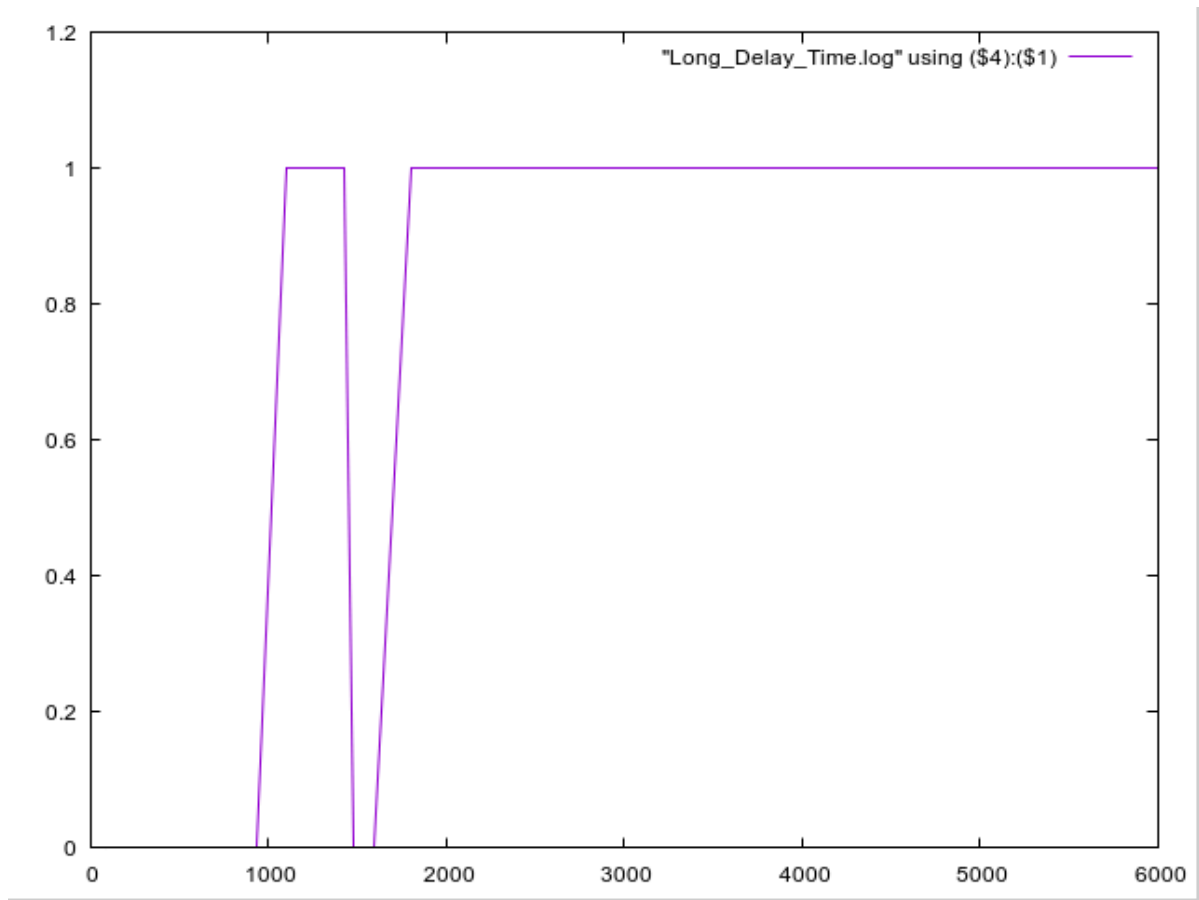
```
globref longdaytime = 200
```

После запуска программы на выполнение в каталоге с кодом программы появится файл Long_Delay_Time.log (рис. [-@fig:015])



Мониторинг параметров моделируемой системы

С помощью gnuplot можно построить график (рис. [-@fig:016]), демонстрирующий, в какие периоды времени значения задержки в очереди превышали заданное значение 200.



Выводы

В процессе выполнения данной лабораторной работы я реализовала модель системы массового обслуживания $M|M|1$ в CPN Tools.

...